

Unrolled List
–Display List
–Time Complexity Analysis

Program:

```
void displayList() {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    cout << "\n--- Unrolled Linked List Content ---\n";
    while (temp != NULL) {
        cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
        for (int i = 0; i < temp->numElements; i++) {
            cout << temp->elements[i];
            if (i < temp->numElements - 1) {
                cout << ", ";
            }
        }
        cout << "]\n";

        temp = temp->next;
        nodeCount++;
    }
    cout << "-----\n";
}

.....

    case 3:
        displayList();
        break;
```

Time Complexity Analysis

- *n*: The total number of elements currently stored across the entire list.
- *N*: The total number of node blocks currently chained together in the list.
- *B*: The maximum capacity of each node (**MAX_ELEMENTS**), which is fixed at 4.

Because each node holds up to *B* elements, the number of nodes is bounded by

$N = \left\lceil \frac{n}{B} \right\rceil$. Since *B* is a constant (4), *N* scales directly and proportionally

with *n*.

Therefore, each node holds up to 4 elements, *N*

$\approx \left\lceil \frac{n}{4} \right\rceil$, meaning *N* scales proportionally with *n*.

Therefore, $O(N)$ is mathematically equivalent to $O(n)$ i.e $O(N) \equiv O(n)$.

1. Best Case Time Complexity: $O(1)$

The absolute best – case scenario occurs when the function

is called on an empty data structure.

- **Condition**: if (**head** == **NULL**) evaluates to true.
- **Execution**: The program prints The list is empty. and immediately executes a **return**; statement, exiting the function.
- **Cost**: Because it performs a single check and terminates

without traversing any loops, it executes

in strictly constant time: **$O(1)$** .

2. Worst & Average Case Time Complexity: $O(n)$ (or $O(N)$)

For the `displayList` function, the worst – case and average – case time complexities are identical.

There is no probability or early exit to average out; the function is forced to perform a complete, exhaustive traversal of the unrolled linked list(When not Empty).

A. Node Traversal (Outer Loop)

```
while (temp != NULL) {
    cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
    ...
}
```

- The outer loop **while (temp ≠ NULL)** visits every single node from **head** to **NULL**.

```
cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
```

- *Inside this loop, it prints the node header information*
(e.g., Node 1 (4/4 elements)) takes $1 + 1 + 1 \dots N = N$ times.

```
cout << "]\n";
```

- Prints ``` and goes to new line takes $1 + 1 + 1 \dots N = 'N'$ times.

```
temp = temp->next;
```

- **Temp = Temp → Next**, this assignment runs, till **Temp = NULL** hence : $1 + 1 + 1 \dots N = N$ times.

```
nodeCount++;
```

- Increments the **nodeCount** till nodecounts, runs: $1 + 1 + 1 \dots N = N$ times.

That is all these **4** line of codes runs inside **while (temp ≠ NULL)** : **4N** times, where, **N** is the total number of node blocks currently chained together in the list.

- **Cost**: Let c_n be the cost of hopping to the next node and printing the header. The total traversal cost is $c_n \times N$ or **4N**.

B. Element Display (Inner Loop)

```
for (int i = 0; i < temp->numElements; i++) {  
    cout << temp->elements[i];  
    if (i < temp->numElements - 1) {  
        cout << ", ";  
    }  
}
```

Within each visited node, the inner loop executes. While it runs a maximum of **B** times per individual node, across the entire execution of the program, it must process every single element in the data structure exactly once.

```
cout << temp->elements[i];
```

Prints the actual integer value. Since it prints every active element across the entire list, this takes $1 + 1 + 1 \dots n = n$ times.

Where, **n**, is the total number of elements currently stored across the entire list.

Note : we are printing like:

Node 1 (Elements 1 through 4)

- *Target is 1st element: Prints 1st element.*
- *Target is 2nd element: Prints 2nd element.*
- *Target is 3rd element: Prints 3rd element.*
- *Target is 4th element: Prints 4th element.*

Node 2 (Elements 5 through 8)

- *Target is 5th element: Prints 5th element.*
- *Target is 6th element: Prints 6th element.*
- *Target is 7th element: Prints 7th element.*
- *Target is 8th element: Prints 8th element.*

Hence , here `n` elements represents total `n` elements of `N` node blocks.

```
if (i < temp->numElements - 1)
```

Evaluates whether the current element is the last one in the node to determine if a comma is needed. This condition is checked for every single element, taking $1 + 1 + 1 \dots n = n$ times.

```
cout << ", ";
```

Let's represent the number of elements in each node as variables:

- *Node 1 has k_1 elements.*
- *Node 2 has k_2 elements.*
- *Node N has k_N elements.*

The total number of elements in the entire list is:

$$n = k_1 + k_2 + \dots + k_N$$

Now, let's calculate the total number of commas printed based on the code's rule (print commas for all elements minus 1 per node):

- Commas in Node 1 = $k_1 - 1$
- Commas in Node 2 = $k_2 - 1$
- Commas in Node N = $k_N - 1$

To find the global total of commas printed, we add them all together:

$$\text{Total Commas} = (k_1 - 1) + (k_2 - 1) + \dots + (k_N - 1)$$

We can rearrange the terms by grouping the k variables together and the 1s together:

$$\text{Total Commas} = (k_1 + k_2 + \dots + k_N) - (1 + 1 + \dots \text{repeated } N \text{ times})$$

Substitute n back into the first group, and simplify the second group:

$$\text{Total Commas} = n - N$$

That is, all these lines of code run inside the inner for loop a total of:

$$n + n + (n - N) = 3n - N \text{ times,}$$

where n is the total number of elements and N is the total number of nodes.

Cost: Let c_p be the combined constant cost of checking and printing a single element. The total element display cost is $c_p \times n$ or roughly $3n$.

$$T(n) = \text{Total Traversal Cost} + \text{Total Element Display Cost}$$

$$T(n) = (c_n \times N) + (c_p \times n)$$

or, represented by the exact execution counts:

$$T(n) = 4N + (3n - N) = 3N - 3n$$

$$\text{Where, } c_n = 3 \text{ and } c_p = 3.$$

To convert this into asymptotic Big – O notation, we drop the constant multipliers (3):

$$T(n) = O(N) + O(n)$$

As, $N \approx \left\lceil \frac{n}{B} \right\rceil$, here, $N \approx \left\lceil \frac{n}{4} \right\rceil$, meaning:

"N "scales directly and proportionally with "n".

*Therefore, **$O(N)$** is mathematically equivalent to **$O(n)$** , making the final Worst & Average Case Time Complexity definitively **$O(n)$** (or **$O(N)$**).*

***Note:** Fundamentally, an Unrolled Linked List still relies on sequential searching. Therefore, the Worst and Average Case Time Complexity always resolves to **$O(n)$** also, **$O(N)$** .*

There is no real Average Case here as:

Whether the list is completely full or mostly empty, if it has n elements, the computer must traverse N nodes and print n elements.

*Hence : Worst = Average = **$O(n)$** or **$O(N)$** .*
